Politechnika Wrocławska

Ochrona Systemów Operacyjnych – Projekt I

System alertów o podejrzanej aktywności

Skład grupy:

- Adam Zawisza: 272292@student.pwr.edu.pl
- Mateusz Logoń: 272225@student.pwr.edu.pl
- Mikołaj Swakoń: 272163@student.pwr.edu.pl
- Piotr Kupczyk: 272180@student.pwr.edu.pl

Data oddania: 29.10.2026

Spis treści

1 Wstęp **3**

2 Opis zagadnienia **3**
2.1 Monitorowanie zdarzeń logowania . . . 3
2.2 Rozpoznawane zagrożenia . . . 3
2.3 Monitorowane zdarzenia systemu Windows . . . 3
2.4 Założenia dotyczące bezpieczeństwa . . . 4

3 Metody i narzędzia **4**
3.1 Założenia projektu . . . 4
3.2 Metoda analizy . . . 5
3.3 Reguły wykrywania . . . 5
3.4 Stosowane technologie . . . 5
3.5 Obsługa alertów . . . 5
3.6 Przetwarzanie strumienia i czas . . . 6
3.7 Metoda detekcji i kryteria interpretacji . . . 6

4 Implementacja **6**
4.1 Architektura rozwiązania . . . 6
4.2 Model danych i identyfikacja rekordów . . . 7
4.3 Konfiguracja środowiska i budowanie . . . 8
4.3.1 Wymagania środowiskowe . . . 8
4.3.2 Przygotowanie audytu Windows . . . 8
4.3.3 Poświadczenia i instalacja . . . 9
4.4 Parametry konfiguracji . . . 9
4.5 Implementacja analizy logów . . . 10
4.5.1 Pobieranie i odtwarzanie historii . . . 10
4.5.2 Parsowanie XML i normalizacja . . . 11
4.5.3 Historia, czas i rekordy poza kolejnością . . . 11
4.6 Implementacja reguł detekcji . . . 11
4.6.1 Sukces po niepowodzeniach . . . 12
4.6.2 Przykład analityczny . . . 12
4.7 Alerty, zapis dowodów i wysyłanie . . . 12
4.7.1 Dobór materiału dowodowego . . . 12
4.7.2 Format ostrzeżeń i wiadomości . . . 13
4.7.3 Trwała kolejka i ponowienia . . . 13
4.7.4 Granice niezawodności . . . 14
4.8 Eksploatacja i diagnostyka . . . 14
4.9 Testy i wyniki . . . 15
4.9.1 Wyniki projektu . . . 15
4.9.2 Weryfikacja wykonana podczas opracowania . . . 16
4.9.3 Plan uzupełniającej walidacji funkcjonalnej . . . 17
4.9.4 Przypadki brzegowe do odtworzenia w testach rdzenia . . . 18
4.9.5 Gromadzenie dowodów i raportowanie wyników . . . 18
4.9.6 Proponowane pomiary efektywności . . . 18

5 Podsumowanie i wnioski **19**
5.1 Ocena realizacji celów 19
5.2 Ocena efektywności i ograniczenia 19
5.3 Możliwe kierunki rozwoju 19

Bibliografia **20**

A Mapa źródeł i zawartość paczki **21**
A.1 Mapa implementacji 21
A.2 Dostarczone pliki 21

B Przykłady bezpośrednio z kodu **22**
B.1 Ograniczenia parsera XML 22
B.2 Utrwalenie zapisu przed podmianą pliku 22
B.3 Konfiguracja detekcji projektu 22

1 Wstęp

Celem niniejszego projektu jest opracowanie systemu alertów o podejrzanej aktywności. Ma on umożliwić użytkownikowi detekcję podejrzanych aktywności w monitorowanym systemie. Rozwiązanie ma analizować systemowe dzienniki zdarzeń i identyfikować wzorce mogące wskazywać na próbę nieuprawnionego dostępu. Do wymaganych funkcji systemu należą:

- Analiza logów zdarzeń systemowych.
- Wykrywanie zbyt wielu nieudanych logowań z jednego IP/konta.
- Wysyłanie powiadomienia e-mail lub zapis do logu ostrzegawczego.

Projekt zakłada stworzenie działającej w tle usługi dla systemu Windows, która będzie stale monitorować dzienniki zdarzeń i automatycznie reagować na podejrzane próby logowania. Po wykryciu zagrożenia system ma zapisywać związane z nim informacje na dysku oraz wysyłać wiadomość ostrzegawczą na wskazany adres e-mail.

2 Opis zagadnienia

2.1 Monitorowanie zdarzeń logowania

Dziennik zdarzeń systemu Windows zawieraja informacje dotyczące aktywności użytkowników i sesji logowania. Analiza tych danych może pozwolić na wyszukanie wzorców odbiegających od typowego zachowania, które to mogą świadczyć o próbach uzyskania nieuprawnionego dostępu do istotnych kont użytkowników. System alertów o podejrzanej aktywnoci ma analizować te zdarzenia w krótkich oknach czasowych i powiadamiać o nich użytkownika.

2.2 Rozpoznawane zagrożenia

Projekt zakłada wykrywanie następujących zagrożeń:

- **Ataki brute-force** - system reaguje na serię nieudanych logowań na to samo konto.
- **Skanowanie kont** - system wyłapuje sytuacje, gdzie z jednego adresu IP ktoś próbuje włamać się na wiele różnych kont.
- **Ataki rozproszone** - system alarmuje, gdy jedno konto jest atakowane z wielu różnych miejsc jednocześnie.
- **Przełamanie zabezpieczeń** - system reaguje na sytuację, w której po serii błędów następuje nagle udane logowanie.

2.3 Monitorowane zdarzenia systemu Windows

W projekcie przewidziano wykorzystanie wybranych identyfikatorów zdarzeń systemowych.

Tabela 1: Monitorowane zdarzenia systemu Windows uwzględnione w projekcie

ID	Znaczenie	Rola w projekcie
4624	Utworzenie sesji po udanym logowaniu	Może uruchomić regułę „sukces po niepowodzeniach”.
4625	Nieudane logowanie	Podstawowe dane do wykrywania podejrzanych wzorców.
4634	Zakończenie sesji logowania	Dodatkowy kontekst dla alertu.
4647	Wylogowanie zainicjowane przez użytkownika	Dodatkowy kontekst dla alertu.
4672	Przyznanie specjalnych uprawnień nowej sesji logowania	Dodatkowy kontekst dla alertu.

2.4 Założenia dotyczące bezpieczeństwa

W projekcie wykorzystano następujące zabezpieczenia systemu:

- **Szyfrowanie haseł** - hasła będą szyfrowane i sprzętowo przypisane do konkretnej maszyny, przez co nie będzie się ich dało wykraść poprzez skopiowanie plików.
- **Zabezpieczenie dostępu** - usługa będzie działać na bezpiecznych, systemowych uprawnieniach, a dostęp do jej plików będą mieć tylko administratorzy.
- **Gwarancja dostarczenia alertu** - ww przypadku awarii, system nie gubi powiadomień, ponawia wysłanie maila do skutku.
- **Ochrona przed spamem** - system będzie zapisywał wysłane powiadomienia, przez co w przypadku restartu maszyny nie zaleje skrzynki powielonymi mailami.

3 Metody i narzędzia

3.1 Założenia projektu

Projekt zakłada wykrywanie następujących zagrożeń:

- Systemem będzie działająca w tle usługa dla systemu Windows, stworzona do ciągłego monitorowania bezpieczeństwa.
- Głównym zadaniem systemu będzie automatyczne wyłapywanie podejrzanych i nietypowych prób logowania do systemu.
- System będzie aplikacją, która na żywo analizuje systemowe dzienniki zdarzeń, sprawdzając je pod kątem potencjalnych ataków.
- Po wykryciu zagrożenia system będzie zapisywał dowody na dysku i wysyłał wiadomość ostrzegawczą na wskazany adres e-mail.

3.2 Metoda analizy

Podstawową metodą będzie ciągła analiza dzienników zdarzeń systemu Windows. System ma grupować aktywność użytkowników i adresów IP w krótkich oknach czasowych, a następnie sprawdzać, czy liczba i układ prób logowania odpowiadają zdefiniowanym regułom wykrywania podejrzanej aktywności.

3.3 Reguły wykrywania

Planowane reguły obejmują:

- wykrywanie serii nieudanych logowań na to samo konto,
- wykrywanie prób logowania na wiele kont z jednego adresu IP,
- wykrywanie prób logowania na jedno konto z wielu różnych miejsc,
- wykrywanie udanego logowania po serii nieudanych prób.

Analizator wykorzystuje okno detekcji wynoszące 5 minut. Alert jest generowany po wykryciu co najmniej 5 nieudanych logowań, przy czym reguły uwzględniają również próby na co najmniej 3 różnych kontach lub z co najmniej 3 różnych adresów IP. Adresy źródłowe są odczytywane z danych zdarzeń Windows, walidowane i normalizowane, a nieprawidłowe wartości są pomijane. System analizuje zdarzenia nieudanego logowania (ID 4625) i może wykrywać również udane logowanie następujące po serii nieudanych prób.

3.4 Stosowane technologie

Tabela 2: Technologie i narzędzia wykorzystane w realizacji projektu

Narzędzie / technologia	Zastosowanie / wersja
C# i .NET	`LangVersion=latest`; SDK bazowe `10.0.100`, `rollForward=latestFeature`
`Oso.Core`	`net10.0`; modele, parser, analiza i raportowanie
`Oso.Service`	`net10.0-windows`; `EnableWindowsTargeting=true`
Hosting / WindowsServices	`Microsoft.Extensions.Hosting` i wariant WindowsServices: 10.0.0
Windows Event Log	`System.Diagnostics.EventLog` 10.0.0
Ochrona poświadczeń	`System.Security.Cryptography.ProtectedData` 10.0.0
SMTP i MIME	MailKit 4.18.1; MimeKit jako zależność przechodnia
Kryptografia PKCS	Bezpośrednie przypięcie `System.Security.Cryptography.Pkcs` 10.0.13
PowerShell	Skrypty instalacji dla Windows PowerShell 5.1
Git i pliki blokad	Identyfikacja wersji; `packages.lock.json` w obu projektach

3.5 Obsługa alertów

Po wykryciu podejrzanej aktywności system ma zapisywać informacje stanowiące dowód zdarzenia na dysku i wysyłać ostrzeżenie na wskazany adres e-mail. W razie awarii, system ponawia wysyłanie aż do skutku oraz rejestruje wysłanie powiadomień w celu ograniczenia duplikatów oraz ograniczenia spamu.

3.6 Przetwarzanie strumienia i czas

System łączy odczyt historyczny z subskrypcją nowych zdarzeń. Kolektor uruchamia subskrypcję przed odtworzeniem historii, dzięki czemu zdarzenia przychodzące w trakcie odczytu trafiają do bufora. Analizator usuwa nakładające się rekordy na podstawie ich tożsamości. Kolejki o pojemności 4096 rekordów czekają na zwolnienie miejsca; nie odrzucają świadomie rekordów po zapełnieniu. To ogranicza liczbę obiektów w kolejkach, ale nie wyznacza stałego limitu pamięci całej aplikacji.
Reguły wykorzystują czas zdarzenia, nie czas pobrania rekordu. Dla zdarzenia w chwili t okno obejmuje oba końce przedziału:

$$W_t = \{e : \ e.\text{EventId} = 4625 \ \wedge \ t - 5\,\text{min} \leq e.\text{Timestamp} \leq t\}. \tag{1}$$

Znacznik `DetectedAt` oznacza czas przetwarzania. Stare rekordy odtwarzane przy starcie mogą więc mieć czas wyzwolenia wcześniejszy niż czas detekcji. Przechowywanie historii w pamięci jest ograniczone domyślnie do dziesięciu minut według czasu przetwarzania.

3.7 Metoda detekcji i kryteria interpretacji

Progi są stałe i konfigurowalne: pięć niepowodzeń, trzy różne konta lub trzy różne IP. Zmiana progu zmienia czułość i liczbę alarmów, lecz w ramach projektu nie przeprowadzono eksperymentu, który uzasadniałby optymalność tych wartości. Nie ma list wyłączeń kont i IP, filtrowania po typie logowania ani wygaszania alarmów w postaci cooldown.
Ocena poprawności funkcjonalnej powinna sprawdzić granice progów, granice czasu, deduplikację i zachowanie przy awarii. Ocena skuteczności bezpieczeństwa wymaga dodatkowo opisanych danych referencyjnych, ze wskazaniem rzeczywistych nadużyć i legalnych zdarzeń. Bez nich nie można obliczyć wiarygodnego odsetka fałszywych alarmów, czułości ani precyzji.

4 Implementacja

4.1 Architektura rozwiązania

W kodzie zachowano oznaczenia techniczne wywodzące się ze skrótu kursu. Są to identyfikatory projektów, usługi, konfiguracji i plików, a nie nazwa własna systemu. Rozwiązanie ma dwa projekty. `Oso.Core` zawiera model zdarzenia, parser, analizator, formatowanie i trwałe raportowanie. `Oso.Service` dodaje zależne od Windows wejście/wyjście, konfigurację, integrację z usługami systemowymi i adapter SMTP. Raportowanie w rdzeniu korzysta z interfejsów `IWarningSink` oraz `IEmailSink`.



Rysunek 1: Przepływ danych od Security do ostrzeżeń i e-maila. Opracowanie na podstawie kodu; nie jest to zrzut uruchomionego systemu.

`Program.cs` konfiguruje hosta i rejestruje wspólne instancje obiektów. `MonitorWorker` uruchamia trzy współpracujące pętle: zbieranie, analizę i dostarczanie. Tylko jeden konsument wywołuje analizator; jego słownik historii nie wymaga wówczas równoległej obsługi. Niezależna pętla dostarczania pozwala analizować kolejne zdarzenia podczas oczekiwania na SMTP. Przy awarii dowolnej pętli pozostałe są anulowane, a usługa kończy pracę z błędem zamiast działać częściowo.

Tabela 3: Odpowiedzialności głównych klas

Klasa / plik	Odpowiedzialność
`WindowsCollector`	Subskrypcja zdarzeń, odtworzenie historii, przekazanie rekordów
`EventParser`	XML do znormalizowanego modelu, zachowanie oryginału
`LoginAnalyzer`	Historia, deduplikacja, warunki reguł i wybór dowodów
`FileOutbox`	Utrwalenie alertu i postępu dostarczania w JSON
`DurableReporter`	Ostrzeżenia, e-mail, ponowienia i usuwanie starych znaczników
`WindowsWarningSink`	Wpis Warning z Event ID 1000
`GmailEmailSink`	Odszyfrowanie hasła, MIME, TLS, uwierzytelnienie i SMTP
`MonitorWorker`	Koordynacja trzech pętli, czyszczenie historii przy bezczynności

4.2 Model danych i identyfikacja rekordów

`LoginEvent` przenosi: `EventId`, `RecordId`, `Timestamp`, `Computer`, `Channel`, `Account`, `Sid`, `SourceIp`, `LogonType`, `LogonId`, słownik `Fields` i oryginalny `Xml`. Dane znormalizowane służą porównywaniu, a pola oryginalne umożliwiają późniejszą analizę szczegółów, np. statusu błędu i stacji roboczej.
Tożsamość rekordu ma postać:

```
Computer|Channel|RecordId|Timestamp:O
```

Znacznik czasu pomaga rozróżniać ponownie użyte numery rekordów po wyczyszczeniu dziennika. `SecurityAlert.IdFor` oblicza SHA-256 z bajtów UTF-8 tej tożsamości i zapisuje 64 znaki szesnastkowe. To identyfikator zdarzenia wyzwalającego, nie podpis cyfrowy lub suma integralności całego pakietu dowodowego. Zmiana konfiguracji reguł nie zmienia ID tego samego rekordu.
`RuleMatch` zawiera nazwę reguły oraz liczbę niepowodzeń, kont i IP. `SecurityAlert` zawiera ID, czas detekcji, rekord wyzwalający, wszystkie dopasowania i listę dowodów. Jedno zdarzenie może spełnić kilka reguł, lecz tworzy jeden alert z kilkoma objaśnieniami.

4.3 Konfiguracja środowiska i budowanie

4.3.1 Wymagania środowiskowe

Docelowe uruchomienie wymaga Windows, dostępu konta usługi do Security, działającego audytu oraz możliwości zapisu w katalogu danych i dzienniku ostrzeżeń. Kod wejściowy odrzuca uruchomienie usługi poza Windows. Projekt można budować w systemie Linux lub Windows; publikacja jest przeznaczona dla `win-x64`. Nie dostarcza osobnej publikacji ARM64 ani instalatora MSI.
Do budowania potrzebny jest SDK .NET 10 zgodny z `global.json`. Odtwarzanie i kompilację projektu wykonuje się następująco:

```
dotnet restore src/Oso.Service/Oso.Service.csproj --locked-mode
dotnet build src/Oso.Service/Oso.Service.csproj -c Release --no-restore
```

Aktualny skrypt publikacji nie uruchamia testów. W Windows PowerShell polecenie:

```
.\scripts\Publish.ps1
```

wywołuje w istocie:

```
dotnet publish .\src\Oso.Service\Oso.Service.csproj -c Release -r win-x64 --
    self-contained true -p:RestoreLockedMode=true -o .\artifacts\service
```

Pakiet zawiera środowisko uruchomieniowe .NET, więc docelowa maszyna nie wymaga osobnej instalacji runtime. Wyniki publikacji nie są dołączone do dokumentacji jako gotowy program.

4.3.2 Przygotowanie audytu Windows

Zgodnie z instrukcją konfiguracji projektu w `secpol.msc`, w zaawansowanych zasadach audytu logowania/wylogowania, należy włączyć:

- *Audit Logon*: sukces i niepowodzenie;
- *Audit Logoff*: sukces;
- *Audit Special Logon*: sukces.

Nazwy w polskiej wersji systemu są zlokalizowane. Zasady domenowe mogą zastępować lokalne. Sprawdzenie efektywnej konfiguracji i ostatnich rekordów:

```
auditpol /get /category:*
Get-WinEvent -FilterHashtable @{
  LogName = 'Security'
  Id = 4624,4625,4634,4647,4672
} -MaxEvents 20 | Select-Object TimeCreated,Id,RecordId,MachineName
```

Włączenie polityki nie stanowi dowodu, że konkretny scenariusz wygeneruje wszystkie ID i pola. Trzeba potwierdzić obecność rekordów dla użytego sposobu logowania.

4.3.3 Poświadczenia i instalacja

Skrypty instalacyjne należy uruchamiać jako administrator w Windows PowerShell 5.1. Instalacja wymaga opublikowanego `Oso.Service.exe`. Użytkownik tworzy hasło aplikacji Gmail; wymaga to weryfikacji dwuetapowej i dostępności tej funkcji dla danego konta. Ograniczenia konta organizacyjnego lub programu ochrony mogą uniemożliwić jej użycie. Kod nie implementuje OAuth.

```
.\scripts\Set-GmailCredential.ps1
.\scripts\Install-Service.ps1 -GmailAddress 'monitor@gmail.com' -Recipient '
   alerts@example.com'
```

Adresy powyżej są przykładami z README, nie danymi faktycznego wdrożenia. Hasło wprowadza się w ukrytym monicie skryptu, bez umieszczania go w poleceniu lub konfiguracji JSON. Katalog danych jest oznaczany plikiem `.oso-state-directory`; skrypt odmawia przejęcia obcego niepustego katalogu i przypadkowego nadpisania istniejącego hasła. Celowa rotacja używa `-ReplaceExisting`.

Tabela 4: Domyślne nazwy i lokalizacje wdrożenia

Element	Wartość
Pliki programu	`C:\ProgramFiles\OSO`
Stan i poświadczenia	`C:\ProgramData\OSO`
Kolejka trwała	`C:\ProgramData\OSO\outbox`
Zaszyfrowane hasło	`C:\ProgramData\OSO\gmail-password.bin`
Nazwa usługi / źródło logu	`OsoLoginMonitor`
Dziennik ostrzeżeń	`OSO-Warnings`
Tożsamość usługi	`LocalSystem`
Lokalne nadpisania	`appsettings.Local.json` przy pliku wykonywalnym

Instalator sprawdza adresy e-mail, obecność programu i poświadczeń, brak istniejącej usługi oraz brak lokalnej konfiguracji w katalogu wejściowym publikacji. Nie nadpisuje niepustego katalogu instalacyjnego. Ustawia ACL, kopiuje pliki, zapisuje lokalne ścieżki i adresy, rejestruje źródło Event Log i uruchamia walidację aplikacji. Dopiero potem tworzy usługę o starcie automatycznym, konfiguruje restart po 15, 30 i 60 sekundach oraz uruchamia usługę. Licznik awarii resetowany jest po 86400 sekundach.
Instalator nie jest transakcyjny. Błąd po skopiowaniu plików może pozostawić katalog, konfigurację lub źródło logu, nawet jeśli usługa nie została utworzona. Ponowienie może wymagać zbadania częściowego stanu i przygotowania pustego katalogu instalacyjnego. Dokumentacja nie potwierdza automatycznego rollbacku.

4.4 Parametry konfiguracji

Host czyta standardową konfigurację .NET, następnie opcjonalny `appsettings.Local.json`, a na końcu zmienne środowiskowe z prefiksem `OSO_`. Lokalny plik i ten prefiks są ostatnimi jawnie dodanymi źródłami w kodzie. Obiekty opcji są wiązane jednokrotnie;

zmiana ustawień wymaga restartu. Katalog treści jest ustalony obok pliku wykonywalnego, niezależnie od katalogu roboczego usługi.

Tabela 5: Sekcja Detection: wartości domyślne i znaczenie

Parametr	Wartość	Znaczenie
`WindowMinutes`	5	Szerokość okna detekcji
`RetentionMinutes`	10	Retencja historii w pamięci
`StartupLookbackMinutes`	10	Zakres odtwarzania przy starcie
`FailureThreshold`	5	Minimalna liczba błędów
`DistinctAccountThreshold`	3	Różne konta dla jednego IP
`DistinctIpThreshold`	3	Różne IP dla jednego konta

Walidacja wymaga dodatniego okna i dodatnich progów. Retencja musi być co najmniej dwukrotnością okna, a zakres odtwarzania nie może być mniejszy od okna ani większy od retencji. Ustawienia nie spełniające warunków przerywają start.

Tabela 6: Reporting i Smtp: wartości domyślne

Parametr	Wartość / ograniczenie
`Reporting.LogName`	`OSO-Warnings`; nie może być `Security`
`Reporting.SourceName`	`OsoLoginMonitor`
`Reporting.DataDirectory`	`%ProgramData%\OSO`; po rozwinięciu ścieżka absolutna
`Reporting.RetryPollSeconds`	5; minimum 1 sekunda
`Smtp.Host` / `Smtp.Port`	`smtp.gmail.com` / 587
`Smtp.Username`	Puste; wymagany poprawny adres nadawcy
`Smtp.Recipient`	Puste; wymagany poprawny adres odbiorcy
`Smtp.PasswordFile`	`%ProgramData%\OSO\gmail-password.bin`

`Smtp.Port` może mieścić się od 1 do 65535. Port 465 wybiera TLS od początku połączenia, wszystkie pozostałe wymagają STARTTLS. Domyślny próg logowania to Information, także dla Event Log. Przykładowe nadpisanie progu błędów: zmienna `OSO_Detection__FailureThreshold`. Samo ustawienie zmiennej w interaktywnej powłoce nie zapewnia przekazania jej już działającej usłudze.

4.5 Implementacja analizy logów

4.5.1 Pobieranie i odtwarzanie historii

`WindowsCollector` tworzy filtr XPath obejmujący pięć obsługiwanych ID. Nowe rekordy trafiają do wewnętrznego kanału `live`, pojemność 4096, z wieloma potencjalnymi producentami i jednym czytelnikiem. Po włączeniu `EventLogWatcher` kolektor odczytuje historię przez `EventLogReader`, od najstarszych rekordów, z filtrem `timediff(@SystemTime)` w milisekundach. Następnie opróżnia bufor zdarzeń bieżących i kontynuuje nasłuch.
Zewnętrzny kanał do analizatora także ma pojemność 4096 i tryb oczekiwania przy zapełnieniu. Callback Windows synchronicznie czeka na miejsce w buforze. Takie hamowanie producenta nie jest dowodem odporności na dowolne obciążenie: ograniczenia wydajności, pamięci i zachowania źródła pod presją nie były zmierzone.

Błąd subskrypcji kończy kanał z wyjątkiem i zatrzymuje potok. Pojedynczy niepoprawny XML lub wartość powoduje ostrzeżenie diagnostyczne i pominięcie rekordu. Uchwyty rekordów są zwalniane po skopiowaniu XML. Projekt nie implementuje trwałego bookmarka ani kursora Security; przerwa dłuższa od skonfigurowanego odtwarzania może pozostawić nieprzeanalizowane zdarzenia.
Odtwarzanie nie zapewnia pełnego kontekstu dla zdarzeń z samego początku pobieranego zakresu: dla pierwszego rekordu potrzebne błędy mogą leżeć jeszcze przed granicą replay. Ponadto długie opóźnienie odczytu lub kolejki może spowodować odrzucenie rekordów, które w chwili analizy są już starsze niż retencja. Zakres czasowy odtwarzania nie jest więc gwarancją pełnego odzyskania każdego alertu z tego okresu. To ograniczenie wynika z zestawienia filtra kolektora i reguł wygaszania historii.

4.5.2 Parsowanie XML i normalizacja

`EventParser.Parse` używa przestrzeni nazw dokumentu oraz nazw pól `EventData/Data`, zamiast lokalizowanego komunikatu tekstowego Windows. Wymaga ID, numeru rekordu, czasu, nazwy komputera i kanału. Nie obsługuje dowolnego formatu logu tekstowego lub CSV. DTD jest zabronione, resolver zewnętrzny wyłączony, a maksymalna długość dokumentu wynosi $4 \cdot 1024 \cdot 1024$ znaków.
Pola nieobecne, puste lub równe `-` stają się wartościami brakującymi na potrzeby normalizacji. Nazwa konta jest składana z domeną i zamieniana na wielkie litery; SID `S-1-0-0` nie zastępuje brakującej nazwy. Typ logowania jest opcjonalny. Surowy XML i oryginalne pola pozostają w modelu nawet wtedy, gdy część pól nie uczestniczy w regułach.

4.5.3 Historia, czas i rekordy poza kolejnością

Przed analizą rekordu historia jest czyszczona według czasu przetwarzania. Rekord starszy niż retencja lub wybiegający o ponad minutę w przyszłość jest ignorowany. Bieżący rekord zostaje zapisany przed oceną, więc piąte niepowodzenie już spełnia próg pięciu. Deduplikacja w pamięci działa tylko w granicach utrzymywanej historii.
Co około pięć sekund bezczynności worker czyści historię także bez nowego zdarzenia. Rekordy spóźnione liczone są względem własnego czasu, lecz wcześniej wysłane alerty nie są przeliczane. Brak sortującego bufora dla bieżącego strumienia i brak ponownej oceny poprzednich rekordów oznaczają, że wynik dla danych przychodzących poza kolejnością może być niepełny względem pełnej analizy retrospektywnej.

4.6 Implementacja reguł detekcji

Niech a oznacza konto bieżącego zdarzenia, a p jego IP. F_a to rekordy z W_t o koncie a, a F_p to rekordy z W_t o IP p. Brak danego klucza daje pusty zbiór dla odpowiedniej reguły. Liczby różnych kont i IP pomijają wartości brakujące.

Tabela 7: Dokładne warunki reguł dla rekordu 4625 przy ustawieniach domyślnych

Nazwa w kodzie	Warunek
`RepeatedAccountFailures`	$\lvert F_a \rvert \geq 5$
`OneIpMultipleAccounts`	$\lvert F_p \rvert \geq 5$ i co najmniej 3 różne niepuste konta w F_p
`MultipleIpsOneAccount`	$\lvert F_a \rvert \geq 5$ i co najmniej 3 różne niepuste IP w F_a

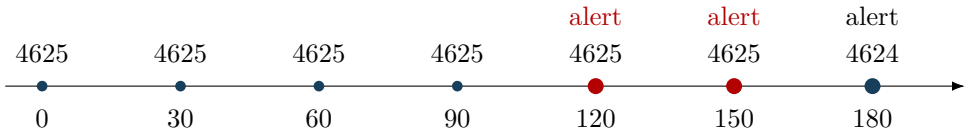
Trzecia reguła spełnia także pierwszą, gdyż obie odnoszą się do pięciu błędów tego samego konta. Oba dopasowania zostają zapisane w jednym alercie. Dla reguły jednego IP liczba błędów obejmuje również rekordy bez konta, ale do progu trzech kont liczą się tylko konta znane. Dla reguły wielu IP analogicznie nieznane IP nie powiększają liczby źródeł.

4.6.1 Sukces po niepowodzeniach

Dla zdarzenia 4624 sprawdzane są te same zbiory wcześniejszych błędów. Dopasowanie otrzymuje przedrostek `SuccessAfter:`, np. `SuccessAfter:RepeatedAccountFailures`. Dla wzorca jednego IP wymagane jest dodatkowo, aby konto udanego logowania występowało wśród kont wcześniejszych prób z tego IP. Sam współdzielony adres nie wystarcza.
Nie wystarczy dowolny pojedynczy błąd przed sukcesem: wcześniejsze niepowodzenia nadal muszą spełniać próg i pozostałe warunki w oknie. Sukces nie zeruje historii ani nie usuwa błędów. Następne udane logowanie w tym samym kwalifikującym się kontekście może więc wygenerować kolejny alert.

4.6.2 Przykład analityczny

Poniższa sekwencja jest obliczeniem z kodu, a nie zarejestrowanym eksperymentem. Zakładamy świeżą historię, brak IP oraz błędy jednego konta w sekundach 0, 30, 60, 90, 120 i 150. Pierwsze cztery rekordy nie wywołają reguły konta; piąty i szósty wywołają po jednym alercie. Sukces tego konta w sekundzie 180 wywoła trzeci alert z przedrostkiem `SuccessAfter:`.



Rysunek 2: Przykład działania progu bez grupowania powiadomień. Liczby oznaczają sekundy od początku syntetycznej sekwencji.

Brak grupowania dotyczy nowych kwalifikujących się zdarzeń, a deduplikacja dotyczy ponownego odczytu tego samego rekordu. Są to odrębne mechanizmy. Powtarzanie nowych prób może wywołać wiele wiadomości, choć żadna nie jest duplikatem identycznego wejścia.

4.7 Alerty, zapis dowodów i wysyłanie

4.7.1 Dobór materiału dowodowego

Analizator zbiera konta i IP związane z rekordem wyzwalającym oraz ze wszystkimi dopasowanymi wzorcami. Z historii wybiera już znane rekordy, które nie są późniejsze niż wyzwalacz i mieszczą się w oknie retencji liczonym wstecz od niego. Dołącza rekordy powiązane przez konto lub IP, a następnie kontekst sesji o tej samej parze komputer–LogonId. Rekord wyzwalający jest włączany jawnie. Wynik jest sortowany po czasie i numerze rekordu.
Tak wybrane dane mogą zawierać 4624, 4625, 4634, 4647 i 4672. Nie wszystkie są przyczyną alarmu; część wyłącznie objaśnia kontekst. Powiązanie przez IP może zebrać zdarzenia różnych użytkowników współdzielących źródło. Przyszłe wylogowanie nie pojawi się w już utrwalonym alercie. Kod nie aktualizuje go późniejszymi dowodami.

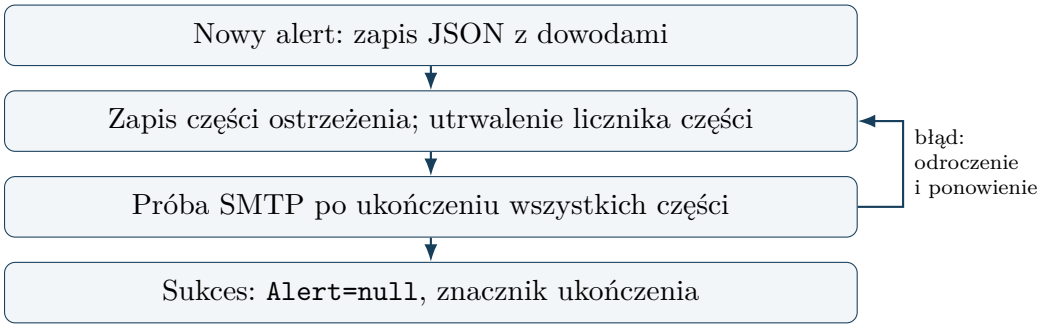
4.7.2 Format ostrzeżeń i wiadomości

`AlertFormatter.Summary` zapisuje ID, czas detekcji UTC, komputer, konto, IP, ID i numer rekordu wyzwalającego, typ logowania, wszystkie nazwy reguł i liczniki oraz liczbę rekordów dowodowych. Zawiera też informację, że alert wskazuje podejrzenie. `EvidenceXml` opakowuje oryginalne XML-e w element `Events`.
Ostrzeżenie zawiera podsumowanie i XML. Duże dane są dzielone na części. Domyślny limit funkcji to 14000 znaków, z rezerwą 160 znaków na ID i numerację. Kod unika podziału pary surogatów UTF-16. Każdy wpis zawiera `Alert<ID>;parti/n`; pojedyncza część nie musi być poprawnym samodzielnym XML-em. Wszystkie części trzeba odtworzyć według ID i kolejności.
Adapter Windows zapisuje wpis typu Warning z ID 1000 przez zarejestrowane źródło `OsoLoginMonitor`. Nie należy utożsamiać tego ID z identyfikatorami zdarzeń logowania w Security. Diagnostyka pracy usługi jest odrębną kategorią wpisów.
Wiadomość MIME ma temat `[OSO]Suspiciousloginactivityon<Computer>`, tekstowe podsumowanie i załącznik `events-<ID>.xml`. `Message-Id` jest deterministyczny: `oso.<id-male-litery>@<host-smtp>`. Ułatwia korelację ponowień, lecz nie wymusza deduplikacji po stronie skrzynki. Klient SMTP ma timeout 30000 ms i zachowuje standardową weryfikację certyfikatu.

4.7.3 Trwała kolejka i ponowienia

Każdy alert jest najpierw zapisywany w `outbox/<ID>.json`. Nazwa ID jest walidowana jako 64 znaki szesnastkowe. Zapis używa pliku tymczasowego, serializacji JSON, opróżnienia bufora na dysk i zastąpienia pliku docelowego. Odczyt obejmuje tylko `*.json`, a nie niedokończone pliki tymczasowe. Semaphore zabezpiecza operacje plikowe wewnątrz procesu; wyłączna blokada `monitor.lock` uniemożliwia równoległe instancje z tym samym katalogiem stanu.
`DeliveryRecord` zawiera `Id`, czas zdarzenia `EventTime`, ładunek `Alert`, liczbę zapisanych części `WarningPartsWritten`, `Attempts`, `NextAttempt` i `CompletedAt`. Przebieg:

1. zapis nowego alertu; istniejący ID w kolejce blokuje ponowne przyjęcie;
2. zapis kolejnych części ostrzeżenia i utrwalanie postępu po każdej części;
3. wysłanie e-maila dopiero po ukończeniu ostrzeżeń;
4. po sukcesie usunięcie ładunku dowodowego z JSON i zapis ukończenia;
5. przy zwykłym błędzie zachowanie ładunku i ustawienie czasu ponowienia.



Rysunek 3: Uproszczony cykl dostarczania i miejsca utrwalania postępu.

Opóźnienie po kolejnych nieudanych próbach wynosi 15, 30, 60, 120, 240, a następnie maksymalnie 300 sekund. Sprawdzenie terminów odbywa się domyślnie co pięć sekund,

więc to nie jest gwarancja wysłania dokładnie po zadanym czasie. Nie ma limitu liczby ponowień ani kolejki odrzuconych wiadomości. Odczyt kolejki porządkuje rekordy po czasie zdarzenia i ID; zaległość może zwiększać opóźnienie.
Po dostarczeniu pozostaje mały znacznik deduplikacyjny. Kod usuwa ukończone rekordy, gdy `EventTime` jest starszy niż 24 godziny; warunek nie jest liczony jako 24 godziny od `CompletedAt`. Długo oczekujący alert może więc utracić znacznik krótko po dostarczeniu. Niewysłane rekordy nie są usuwane przez tę retencję. Windows Event Log pozostaje poza tym mechanizmem.

4.7.4 Granice niezawodności

Ponowienie po zwykłym błędzie SMTP nie zapisuje ponownie ukończonych części ostrzeżenia. Jednak awaria po zapisie zewnętrznym, a przed zapisaniem postępu może powtórzyć część lub wiadomość. Model odpowiada dostarczaniu *co najmniej raz* przy zachowanym stanie i możliwości późniejszego dostarczenia, a nie dokładnie raz. Brak poprawnego JSON lub możliwość zapisu stanu są krytyczne dla pracy potoku; błędu dysku nie wolno traktować jako bezpiecznej utraty nieistotnego pliku.

4.8 Eksploatacja i diagnostyka

`--validate-config` sprawdza kształt opcji, lokalne odszyfrowanie poświadczenia, mapowanie źródła ostrzeżeń i możliwość odczytu Security. Nie uwierzytelnia w Gmailu, nie potwierdza faktycznej generacji 4625, nie uruchamia całej ścieżki kolektora i nie sprawdza dostarczenia do skrzynki.

```
& 'C:\Program Files\OSO\Oso.Service.exe' --validate-config
Get-Service OsoLoginMonitor
Get-WinEvent -FilterHashtable @{
  LogName = 'OSO-Warnings'; Id = 1000
} -MaxEvents 10
Restart-Service OsoLoginMonitor
```

W diagnostyce należy powiązać numer rekordu Security, ID alertu, wszystkie części ostrzeżenia, stan outbox i identyfikator wiadomości. Sam stan Running nie dowodzi działania detekcji. Komunikat `Startupreplaycompleted;monitoringSecuritylog.` wskazuje ukończenie odtwarzania, ale nadal nie stanowi dowodu dostarczenia alertu.

Tabela 8: Typowe objawy i zalecane sprawdzenia

Objaw	Sprawdzenie
Brak rekordów 4625	Efektywna polityka audytu i zachowanie lokalnego mechanizmu logowania
Błąd startu	Wynik walidacji, dostęp do Security, rejestracja źródła, plik hasła i ACL
Ostrzeżenie bez e-maila	Stan outbox, błędy SMTP, wyjście TCP 587/465, konto i aktualność hasła aplikacji
Wiele wiadomości	Różne ID wskazują nowe zdarzenia; ten sam ID może oznaczać ponowienie po awarii
Uszkodzony outbox	Zatrzymanie usługi, zabezpieczenie kopii i analiza; nie usuwać oczekujących dowodów w celu ukrycia błędu
Brak zdarzeń po długiej przerwie	Domyślnie tylko 10 minut odtwarzania, brak trwałego kursora

Deinstalacja przez `scripts/Uninstall-Service.ps1` zatrzymuje usługę i usuwa jej rejestrację, pozostawiając program, konfigurację, poświadczenia, outbox i dziennik. Aktualizacja wymaga zachowania starych binariów i lokalnej konfiguracji, a następnie instalacji do pustego katalogu przy zachowaniu tego samego katalogu danych. Nie wolno zgubić oczekujących rekordów.
Retencję i maksymalny rozmiar dziennika Windows ustala się oddzielnie w systemie. Dane dowodowe oraz wiadomości zawierają konta, IP i XML zdarzeń; zakres dostępu do dziennika i skrzynki wymaga kontroli administratora. Kolejka nie ma limitu wielkości w bajtach ani automatycznego usuwania niewysłanych alertów, więc długotrwała awaria SMTP wymaga monitorowania miejsca na dysku.

4.9 Testy i wyniki

4.9.1 Wyniki projektu

Plik `docs/validation-results.md` opisuje wcześniejszą walidację w środowisku chmurowym Linux z SDK .NET 10.0.402. Nie jest raportem z docelowej maszyny Windows. Zapisane rezultaty przedstawiono w tabeli 9; w trakcie opracowania nie uruchamiano ponownie tych sprawdzeń.

Tabela 9: Historyczne wyniki testów projektu

Sprawdzenie	Wynik deklarowany w notatce
Odtworzenie zależności rozwiązania w trybie blokady	Powodzenie; audyt i weryfikacja podpisów włączone
Kompilacja Release rozwiązania	Powodzenie; zero ostrzeżeń i błędów
Historyczny zestaw testów rdzenia	36 zaliczonych, 0 błędnych, 0 pominiętych
Publikacja Windows x64 self-contained	Powodzenie; plik wykonywalny i runtime
Historyczna wersja skryptu publikacji	Powodzenie; zawierała uruchomienie testów
Parsowanie pięciu skryptów w PowerShell 7.5.4	Powodzenie; nie oznacza wykonania na Windows 5.1
Restore i build po usunięciu testów i solution	Powodzenie dla `Oso.Service.csproj`; bez ostrzeżeń i błędów
Uproszczony skrypt publikacji po porządkach	Powodzenie bez zależności od testów

Notatka zawiera historyczny wiersz mówiący o wykonywaniu testów przez `Publish.ps1`, a następnie wyjaśnienie uproszczenia skryptu. Aktualny skrypt zawiera tylko publikację z blokadą pakietów. Wynik 36 testów nie jest wynikiem testów dostępnych w bieżącej wersji: projekt testowy, solution i zdalny workflow CI usunięto. Brak kodu przypadków, raportu szczegółowego i danych wejściowych uniemożliwia niezależną ocenę ich pokrycia.

4.9.2 Weryfikacja wykonana podczas opracowania

Wykonano przegląd statyczny pełnego drzewa wersji wskazanej we wstępie. Sprawdzono spójność wartości domyślnych, powiązania klas, warunki reguł i kolejność operacji raportowania. Sprawdzono również zgodność strony tytułowej ze źródłem ZIP. Są to wyniki przeglądu dokumentacyjnego, nie pomiar działania usługi.

Tabela 10: Ustalenia przeglądu statycznego

Obszar	Ustalenie
Źródło i filtr zdarzeń	Pięć ID zgodnych z listą parsera
Parametry	Okno 5 min, retencja 10 min, replay 10 min; progi 5/3/3
Brak IP	Reguła konta zachowuje możliwość detekcji
Sukces po błędach	Wymaga nadal kwalifikującego się wzorca; nie zeruje liczników
Wysyłanie	Wszystkie części logu przed e-mailem
Retencja znacznika	24 h od czasu zdarzenia, warunkowo po ukończeniu
Ograniczanie powiadomień	Brak cooldown i grupowania nowych rekordów
Dowody uruchomienia Windows	Nie znaleziono EVTX, rzeczywistych e-maili ani zrzutów testów

Ustalenia są odtwarzalne w `EventParser.cs`, `Models.cs`, `LoginAnalyzer.cs`, `DurableReporter.cs` i adapterach usługi. Manifest w paczce umożliwia sprawdzenie, czy analizowane pliki odpowiadają tej samej wersji.

4.9.3 Plan uzupełniającej walidacji funkcjonalnej

Poniższa macierz rozwija instrukcję `docs/windows-validation.md`. Wszystkie przypadki mają status **NIEWYKONANY w ramach dokumentacji**; kolumna wyniku oczekiwanego opisuje kryterium zaliczenia, nie rezultat. Testy wymagają kontrolowanej VM, kont testowych i działającej sesji administratora. Należy uwzględnić politykę blokady kont.

Tabela 11: Macierz testów do wykonania na Windows

ID	Dane i działanie	Wynik oczekiwany / wymagany dowód
W01	Audyt sukcesów/błędów, instalacja i walidacja konfiguracji	Poprawny wynik walidacji; rekordy Security rzeczywiście powstają; zapis parametrów VM i polityki
W02	Cztery 4625 świeżego konta w pięć minut, bez innych prób	Brak alertu konta; eksport czterech rekordów i przeszukanie ostrzeżeń
W03	Piąty i szósty 4625 tego konta	Dwa różne alerty i dwie wiadomości, z odpowiednim materiałem dowodowym
W04	4624 tego konta, gdy próg błędów nadal mieści się w oknie	Alert z `SuccessAfter: RepeatedAccountFailures`, rekord 4624 i jego typ
W05	Sukces po wygaśnięciu kwalifikujących się błędów	Brak dopasowania sukcesu po błędach; potwierdzone znaczniki czasu
W06	Co najmniej pięć błędów z jednego IP wobec minimum trzech kont	Dopasowanie `OneIpMultipleAccounts`; dowody różnych kont i wspólnego IP
W07	Co najmniej pięć błędów konta z minimum trzech IP	Jeden alert wyzwalacza zawiera regułę konta i `MultipleIpsOneAccount`
W08	Konto bez IP; osobno znane IP i brak konta	Brak IP nie blokuje reguły konta; braki nie zwiększają liczby różnych kluczy
W09	Restart, replay i ponowne odczytanie identycznych rekordów	Już utrwalony ID nie tworzy nowego alertu; nowy kwalifikujący rekord nadal działa
W10	Dostępne 4634/4647/4672 przed następnym wyzwalaczem	Powiązany kontekst w kolejnym alercie; brak dopisywania przyszłych zdarzeń do starego
W11	Czasowe zablokowanie SMTP i późniejsze odblokowanie tylko reguły testowej	Log przed e-mailem, stan oczekujący na dysku, późniejsze dostarczenie bez zwykłego powtórzenia części
W12	Restart z niedostarczoną wiadomością	Zachowany ID, dowody i postęp; dostarczenie po odzyskaniu SMTP
W13	Ponad 10 minut bez zdarzeń, potem nowy alert	Stara historia nie powiększa dowodów; wcześniejsze logi Windows nadal istnieją
W14	Duży alert przekraczający pojedynczą część ostrzeżenia	Wszystkie części mają jeden ID i poprawną numerację; po złożeniu zachowany XML
W15	Uruchomienie drugiej instancji z tym samym stanem; kontrola ACL	Druga instancja odrzucona; zwykły użytkownik nie modyfikuje programu i nie czyta stanu
W16	Kontrolowany błąd stanu / miejsca i sprawdzenie recovery	Jawny błąd i reakcja odzyskiwania; zachowanie kopii dowodów, bez ich cichego usuwania

Testy W06 i W07 wymagają odpowiednich rzeczywistych adresów źródłowych widocznych w XML Windows. Samo lokalne logowanie interaktywne często nie dostarcza IP. Nie należy zastępować takiej walidacji wymyślonymi wpisami w raporcie. Przy testach progów trzeba odizolować przypadki w czasie lub użyć świeżych tożsamości, aby wcześniejsze zdarzenia nie zafałszowały liczników.

4.9.4 Przypadki brzegowe do odtworzenia w testach rdzenia

Jeżeli zestaw automatyczny zostanie przywrócony, powinien objąć: dokładnie 5 minut i przekroczenie o małą jednostkę czasu; odrzucenie rekordu starszego od retencji; przyszły czas ponad minutę; wspólne IP z sukcesem nieatakowanego konta; brak SID/nazwy; IPv4-mapped IPv6; zakazane DTD; nieobsługiwane ID; duplikaty; korelację sesji na różnych komputerach; rekordy poza kolejnością; wznawianie po zapisanej części ostrzeżenia; awarię zapisu JSON i SMTP; granicę 24 godzin znacznika według `EventTime`. Jest to propozycja zakresu, a nie lista istniejących 36 testów.

4.9.5 Gromadzenie dowodów i raportowanie wyników

Dla każdego testu należy zapisać: identyfikator, datę i czas UTC, wersję Windows, commit, konfigurację, kroki, spodziewany rezultat, obserwację i ocenę PASS/FAIL. Wymagane jest powiązanie `Security.RecordId`, `Alert.Id` i wiadomości. Zrzut ekranu jest dodatkiem; eksport rekordów i oryginalny XML pozwalają dokładniej odtworzyć warunki.
Przykładowe polecenia zebrania lokalnych dowodów po utworzeniu dedykowanego katalogu:

```
New-Item -ItemType Directory -Path .\wyniki-oso -Force
auditpol /get /category:* > .\wyniki-oso\audit-policy.txt
Get-Service OsoLoginMonitor |
  Format-List * > .\wyniki-oso\service.txt
Get-WinEvent -FilterHashtable @{
  LogName = 'OSO-Warnings'; Id = 1000
} -MaxEvents 50 | ForEach-Object { $_.ToXml() } |
  Set-Content .\wyniki-oso\warning-records.xml.txt -Encoding UTF8
wevtutil epl OSO-Warnings .\wyniki-oso\oso-warnings.evtx
```

Są to polecenia proponowane w dokumentacji, nie uruchomione na VM. Zbiór wielu XML-i w pliku tekstowym nie jest jednym dokumentem XML. Eksport EVTX oraz ewentualny ograniczony eksport Security należy wykonywać z uprawnieniami administratora i przechowywać zgodnie z dostępem do danych testowych. Nie dołącza się pliku hasła do raportu. Po wykonaniu prób należy zestawić liczbę wejściowych rekordów i alertów z macierzą testów. Dla W03, przy świeżym koncie i wyłącznie sześciu błędach bez innych pasujących wzorców, oczekiwane są dwa alerty błędów; W04 dodaje następny. Brak jakiegokolwiek rzeczywistego wyniku oznacza *niezweryfikowane*, a nie PASS.

4.9.6 Proponowane pomiary efektywności

Nie ma pomiarów wydajności. W przyszłych testach należy zmierzyć czas zdarzenie–detekcja oraz detekcja–potwierdzenie SMTP, zużycie CPU/RAM, wielkość outbox i tempo opróżniania zaległości. Dla oznaczonych danych można wyliczyć precyzję $TP/(TP + FP)$ i czułość $TP/(TP + FN)$, po ustaleniu jednostki oceny. Dostępne materiały nie zawierają wymaganych pomiarów ani etykiet; nie obliczono tych miar. Liczby wiadomości nie należy utożsamiać z liczbą incydentów, gdyż brak grupowania.

5 Podsumowanie i wnioski

5.1 Ocena realizacji celów

W ramach projektu opracowano system do wykrywania podejrzanej aktywności związanej z logowaniem do systemu Windows. Rozwiązanie analizuje zdarzenia z dziennika Security i identyfikuje wzorce takie jak powtarzające się nieudane logowania, próby dostępu do wielu kont oraz logowania z różnych adresów IP. Wykryte zdarzenia mogą generować alerty zapisywane na dysku i przesyłane pocztą elektroniczną. Implementacja obejmuje również mechanizmy deduplikacji zdarzeń i ponawiania wysyłki powiadomień. Zrealizowano wszystkie cele projektu.

5.2 Ocena efektywności i ograniczenia

Skuteczność detekcji zależy od konfiguracji audytu Windows, jakości dostępnych danych i przyjętych progów. Ich optymalność nie została potwierdzona na reprezentatywnym zbiorze danych, dlatego nie można jednoznacznie określić poziomu fałszywych alarmów. Ponadto brak trwałego punktu wznowienia odczytu dziennika może prowadzić do pominięcia części zdarzeń po dłuższej przerwie w działaniu usługi. Pełna ocena niezawodności wymaga dalszych testów w środowisku docelowym.

5.3 Możliwe kierunki rozwoju

Dalszy rozwój powinien obejmować testy w środowisku Windows, dostrojenie progów detekcji na podstawie rzeczywistych danych oraz wprowadzenie trwałego zapamiętywania miejsca odczytu dziennika. Warto również rozważyć grupowanie alertów, ograniczanie nadmiaru powiadomień i rozszerzenie zakresu analizowanych zdarzeń.

Bibliografia

[1] Zespół projektu, *System alertów o podejrzanej aktywności*, projekt wstępny, 6 stron, materiał dostarczony przez użytkownika; kopia: `materials/projekt-wstepny.pdf`.

[2] A. Florek, *Ochrona systemów operacyjnych – zasady zaliczenia*, 2 strony, materiał kursowy dostarczony przez użytkownika; kopia: `materials/wymagania-kursu.pdf`.

[3] Zespół projektu, *Szablon LaTeX dokumentacji projektu z kursu OSO*, dostarczone archiwum ZIP, plik `main.tex`; kopie: `materials/szablon-oryginalny.zip` i `materials/szablon-oryginalny.tex`.

[4] noname6962, *OSO – kod źródłowy i dokumentacja techniczna projektu*, wersja oznaczona commitem `e0d3e05b0c5402e2d73c03b040fe45cbf72edea5`, 14.10.2026, dostęp 28.10.2026. Źródło obejmuje implementację, konfigurację, skrypty instalacyjne oraz materiały dotyczące walidacji. `https://github.com/noname6962/OSO/tree/e0d3e05b0c5402e2d73c03b040fe45cbf72edea5`.

[5] Microsoft Learn, *4624(S): An account was successfully logged on*, dostęp 28.10.2026. `https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4624`.

[6] Microsoft Learn, *4625(F): An account failed to log on*, dostęp 28.10.2026. `https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4625`.

[7] Microsoft Learn, *DataProtectionScope Enum*, zakres CurrentUser i LocalMachine, dostęp 28.10.2026. `https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.dataprotectionscope`.

[8] Google Account Help, *Sign in with app passwords*, dostęp 28.10.2026. `https://support.google.com/accounts/answer/185833`.

A Mapa źródeł i zawartość paczki

A.1 Mapa implementacji

Wszystkie ścieżki poniżej odnoszą się do wersji `e0d3e05`. Numery linii są orientacją dla przeglądu tej konkretnej wersji, a nie deklaracją wykonania testu.

Tabela 12: Miejsca implementacji kluczowych mechanizmów

Plik względem katalogu głównego projektu	Mechanizm
`src/Oso.Core/Models.cs`	Tożsamość, SHA-256 ID, modele, walidacja progów
`src/Oso.Core/EventParser.cs`	ID 15; parser 18–72; konto 46–53; IP 54–60
`src/Oso.Core/LoginAnalyzer.cs`	`Process`: historia, trzy reguły, sukces i dowody
`src/Oso.Core/DurableReporter.cs`	`FileOutbox`: 30–126; dostarczanie: 144–190
`src/Oso.Core/AlertFormatter.cs`	Podsumowanie 9–15; XML 18–19; części 22–39
`src/Oso.Service/WindowsCollector.cs`	Watcher, replay i bufor live
`src/Oso.Service/MonitorWorker.cs`	Trzy pętle, czyszczenie historii, błąd potoku
`src/Oso.Service/Program.cs`	Start, walidacja, blokada instancji i DI
`src/Oso.Service/ServiceOptions.cs`	Reporting i Smtp oraz walidacja
`src/Oso.Service/GmailEmailSink.cs`	DPAPI, format MIME i SMTP/TLS
`src/Oso.Service/WindowsWarningSink.cs`	Warning, ID 1000
`scripts/*.ps1`	Budowanie, ACL, poświadczenia, instalacja i usunięcie usługi

A.2 Dostarczone pliki

```
OSO_LaTeX/
  main.tex                # dokument glowny
  titlepage.tex           # strona tytulowa z ZIP-a
  bibliografia.tex        # bibliografia bez BibTeX/Biber
  sections/               # rozdzialy dokumentacji
  figures/                # edytowalne rysunki TikZ
  excerpts/               # wybrane oryginalne zrodla i konfiguracja
  materials/              # oba PDF-y, oryginalny ZIP i jego main.tex
  reference/OSO/          # 27 plikow projektu
  evidence/               # manifest, oryginalne notatki walidacji
  README.md               # instrukcja kompilacji i zakres dowodow
  build.sh                # lokalna kompilacja latexmk
```

Finalny PDF towarzyszy projektowi. Diagramy są edytowalnymi rysunkami TikZ, a przykłady i scenariusze testowe nie zastępują dowodów z Windows.

B Przykłady bezpośrednio z kodu

Poniższe fragmenty pochodzą z kodu źródłowego projektu. Pełne wybrane pliki umieszczono w `excerpts/`; ich zawartości nie zmieniano.

B.1 Ograniczenia parsera XML

```
        // Do not resolve external XML entities or allow unusually large input
    documents.
        using var text = new StringReader(xml);
        using var reader = XmlReader.Create(text, new XmlReaderSettings
        {
            DtdProcessing = DtdProcessing.Prohibit, XmlResolver = null,
            MaxCharactersInDocument = 4 * 1024 * 1024
        });
```

B.2 Utrwalenie zapisu przed podmianą pliku

```
        var target = PathFor(record.Id);
        var temp = target + ".tmp";
        // Flush before atomic replacement so a restart never observes half a
    JSON record.
        await using (var stream = new FileStream(temp, FileMode.Create,
    FileAccess.Write, FileShare.None))
        {
            await JsonSerializer.SerializeAsync(stream, record, Json, ct);
            await stream.FlushAsync(ct);
            stream.Flush(flushToDisk: true);
        }
        // Readers enumerate only *.json, so an interrupted *.json.tmp write is
     never treated as an alert.
        File.Move(temp, target, overwrite: true);
```

B.3 Konfiguracja detekcji projektu

```
  "Detection": {
    "WindowMinutes": 5,
    "RetentionMinutes": 10,
    "StartupLookbackMinutes": 10,
    "FailureThreshold": 5,
    "DistinctAccountThreshold": 3,
    "DistinctIpThreshold": 3
  },
```